

8086 - Como funciona a pilha ou stack pointer

O que ocorre nos registradores SS, SP e BP no 8086 durante a chamada de uma função?

Durante a chamada de uma função (instrução **CALL**) e a criação do seu quadro de pilha (*stack frame*) no processador Intel 8086, os registradores **SS**, **SP** e **BP** desempenham papéis específicos para organizar a memória e os dados da função.

O registrador SS (Stack Segment) permanece inalterado (a menos que seja uma chamada do tipo *Far*, o que é raro para funções comuns), enquanto **o SP (Stack Pointer) diminui** à medida que novos dados são empilhados, e **o BP (Base Pointer) é atualizado** para apontar para o início do quadro da nova função.

Abaixo está o comportamento detalhado de cada registrador durante o ciclo de vida da chamada:

1. O que acontece com cada registrador

- **SS (Stack Segment):** Armazena o endereço base do segmento de pilha. Durante uma chamada de função interna (*Near CALL*), o **SS não sofre nenhuma alteração**. Ele continua apontando para o mesmo bloco de memória de 64 KB reservado para a pilha.
- **SP (Stack Pointer):** Aponta para o topo atual da pilha. Como a pilha no 8086 cresce para baixo (em direção a endereços de memória menores), **o SP decrementa**.
- No momento exato do **CALL**, o processador subtrai 2 do SP ($SP = SP - 2$) e salva o endereço de retorno (o valor do registrador **IP**) na pilha.
- Se variáveis locais forem criadas ou outros registradores forem salvos (**PUSH**), o SP continuará diminuindo.
- **BP (Base Pointer):** Usado como um ponto de referência fixo dentro da função. Ele **não muda de forma automática pelo hardware**, mas por convenção de software (o prólogo da função), ele é atualizado para salvar o topo da pilha logo após a chamada. Isso permite acessar parâmetros e variáveis locais usando deslocamentos fixos como `[BP+4]` ou `[BP-2]`.

2. A Linha do Tempo na Prática (Prólogo da Função)

Para entender as mudanças, veja o que acontece passo a passo no código Assembly padrão:

```
; --- Programa Principal ---  
PUSH param1      ; SP diminui 2  
CALL minha_funcao ; SP diminui 2 (salva o IP de retorno)
```

```

; --- Dentro da Função (Prólogo) ---
minha_funcao:
    PUSH BP      ; SP diminui 2 (salva o BP da função anterior)
    MOV BP, SP   ; BP AGORA RECEBE O VALOR DE SP. O BP vira a referência
estável.
    SUB SP, 4     ; SP diminui 4 (reserva espaço para 2 variáveis locais de 16
bits)

```

Momento	Registrador SS	Registrador SP	Registrador BP
Antes do CALL	Estável (Aponta para o segmento)	Aponta para o topo atual	Guarda o valor do escopo anterior
Logo após o CALL	Inalterado	Diminui 2 (Contém o endereço de retorno)	Inalterado
Após salvar o BP antigo	Inalterado	Diminui mais 2	Inalterado
Após <code>MOV BP, SP</code>	Inalterado	Estável	Recebe o valor atual de SP
Após criar variáveis locais	Inalterado	Diminui pelo tamanho das variáveis	Estável (Fixo, referenciando o quadro)

3. O Retorno da Função (Epílogo)

Ao final da função, o processo é invertido para restaurar os registradores aos seus valores originais:

1. `MOV SP, BP` liberta o espaço das variáveis locais (SP aumenta).
2. `POP BP` restaura o ponteiro base da função que chamou (SP aumenta 2).
3. A instrução `RET` retira o endereço de retorno da pilha e o joga de volta no `IP` (SP aumenta 2).

Como funciona o registro bp no 8086?

O registrador **BP (Base Pointer)** é um registrador de propósito geral de **16 bits**, mas que possui uma função altamente especializada: ele serve como uma **âncora fixa para acessar dados dentro da pilha** (Stack).

Sem o **BP**, gerenciar parâmetros de funções e variáveis locais em Assembly seria extremamente difícil, pois o topo da pilha (controlado pelo **SP**) muda de lugar a todo momento.

Abaixo está o funcionamento detalhado e as regras de hardware do **BP**:

1. A Regra de Ouro do Hardware (Acesso ao Segmento SS)

No processador 8086, a maioria dos registradores de índice (como `BX`, `SI`, `DI`) aponta, por padrão, para endereços dentro do segmento de dados (**DS**).

O **BP** é uma exceção única: sempre que você usa o `BP` dentro de colchetes para ler ou escrever na memória, o processador assume automaticamente que você está operando no Segmento de Pilha (**SS**).

- `mov ax, [bx]` → Busca o dado no endereço `DS:BX` (Segmento de Dados)
- `mov ax, [bp]` → Busca o dado no endereço `SS:BP` (Segmento de Pilha)

Isso permite que você acesse qualquer posição da pilha diretamente, sem precisar desempilhar (`POP`) nada.

2. Por que o BP é necessário? (O problema do SP)

O registrador `SP` (Stack Pointer) muda constantemente. Se você faz um `PUSH`, o `SP` diminui. Se faz um `POP`, o `SP` aumenta.

Se você tentasse acessar um parâmetro usando o `SP` como referência, o endereço mudaria a cada instrução:

- No início da função, o parâmetro 1 poderia estar em `[sp+4]`.
- Se a função fizesse um `push cx` para salvar um registrador, o mesmo parâmetro 1 passaria a estar em `[sp+6]`.

O **BP** resolve isso funcionando como uma "foto" do `SP` no início da função. Como o `BP` não muda até a função acabar, os parâmetros e variáveis ficam sempre na mesma distância dele.

3. A anatomia do Frame de Pilha com BP

Por convenção de software, o `BP` divide o "quadro da função" (Stack Frame) em duas zonas fáceis de acessar:

- **Deslocamentos Positivos** `[BP + x]`: Acessam os **parâmetros** enviados por quem chamou a função.

- **Deslocamentos Negativos** `[BP - x]`: Acessam as **variáveis locais** criadas pela própria função.

Exemplo Visual na Memória:

Imagine uma função que recebeu dois parâmetros e criou duas variáveis locais. O mapa da pilha atrelado ao **BP** fica assim:

Endereço Alto	-----	
	Parâmetro 2	[BP + 6]
	Parâmetro 1	[BP + 4]
	Endereço de Retorno	[BP + 2]

BP Aponta Aqui	Antigo valor de BP	[BP] <-- Âncora fixa

	Variável Local 1	[BP - 2]
	Variável Local 2	[BP - 4]
Endereço Baixo	-----	<-- Topo atual da pilha (SP)

4. Modos de Endereçamento Suportados

O 8086 permite combinar o **BP** com outros registradores de índice (como **SI** ou **DI**) e constantes para ler arrays ou estruturas alocadas na pilha. Exemplos válidos:

- `mov ax, [bp+4]` (Base + Deslocamento constante)
- `mov ax, [bp+si]` (Base + Índice)
- `mov ax, [bp+di+2]` (Base + Índice + Deslocamento constante)